Professional development for information security specialists

PRACTICAL WORK GUIDE
FOR STUDENTS

course:
“Methods and Principles of Designing Modern Secure Systems. Using Artificial Intelligence Methods for Data Analysis in Information Security Systems”

PRACTICE 2. SECURITY GATE FOR A RELEASE CANDIDATE

SAST, secrets, SBOM, dependencies and container

(65 minutes)

2026

Contents

Scenario and result 3

0. Preparation — 5 minutes 4

1. Normal build — 5 minutes 5

2. SAST: double free — 12 minutes 6

    2.1. Report in SARIF format 7

3. DAST: dynamic analysis with ASAN — 3 minutes 9

4. Search for secrets — 10 minutes 11

5. Image, SBOM and dependencies — 12 minutes 12

6. Container — 9 minutes 13

7. Fixes — 6 minutes 14

    7.1. One place to free memory 14

    7.2. Check of the index 14

    7.3. Container 15

    7.4. Secrets and Git history 15

8. Second Security Gate run — 3 minutes 17

Scenario and result

Your team is preparing the release of `release-agent 2.0`. It is a small C program in a Docker image. You are a member of the Security Gate. Each stage saves a machine-readable report in `/work/reports/`. The release is allowed only after a second check.

You log in to the Ubuntu container once. After that you work in the file `/work/playbook.sh`. Copy the commands from this guide into the function of the matching stage. Put them in place of the `:` line. The source code is in `/work/repository`. The reports are in `/work/reports`.

Table 1 — Route of the practice — 65 minutes

Time	Control	Artifact
5 min	Preparation	working Git repository
5 min	Normal build	test passes
12 min	Clang Static Analyzer	`sast.txt`, `sast.sarif`
3 min	AddressSanitizer	`dast-asan.txt`
10 min	Gitleaks	`gitleaks.json`
12 min	Syft and Grype	`sbom.cdx.json`, `grype.json`
9 min	Trivy	`trivy.json`
6 min	Fixes	fixed repository
3 min	Second Gate run	decision `PASS` or `BLOCK`

0. Preparation — 5 minutes

Open a terminal and run:

```
./lab.sh start
./lab.sh shell
```

Run the commands from the `materials` folder you received. The first command builds one image based on Ubuntu 24.04 and starts it. You do this only once. The second command opens Bash inside the container. On the first start the lab creates `/work/repository` with two commits. It also creates the reports folder and the playbook.

Run all the next commands inside the open container. Check the lab:

```
pwd
tree -a -L 2 /work
clang --version | head -1
gitleaks version
syft version
grype version
trivy --version | head -1
```

The working path is `/work`. Open the template with `nano /work/playbook.sh`. In `/work/repository` you should see two commits and these files: `src/session.c`, `Dockerfile`, `compose.yaml`, `config/app.env`.

1. Normal build — 5 minutes

In the function `baseline()`, replace `:` with this code:

```
cmake -S . -B build -DCMAKE_C_COMPILER=clang
cmake --build build
ctest --test-dir build
./build/release-agent
```

Run:

```
./playbook.sh baseline
```

The build finishes without errors. The test passes. The normal run prints `session closed`. At this point you see no sign of a problem. But the test checks only one path through the program. So it is too early to decide if the release is safe.

2. SAST: double free — 12 minutes

In `scan_sast()`, write:

```
local output
if ! output="$(clang --analyze -Xanalyzer -analyzer-checker=unix.Malloc \
  -Xanalyzer -analyzer-output=text -Iinclude src/session.c 2>&1)"; then
  printf "%s\n" "$output" | tee /work/reports/sast.txt
  echo "SAST stopped: src/session.c does not compile" >&2
  return 1
fi
if [[ -n "$output" ]]; then
  printf "%s\n" "$output" | tee /work/reports/sast.txt
else
  echo "No analyzer findings" | tee /work/reports/sast.txt
fi
clang --analyze -Xanalyzer -analyzer-checker=unix.Malloc \
  -Xanalyzer -analyzer-output=sarif -Iinclude src/session.c \
  -o /work/reports/sast.sarif
```

The first `if` block is for the case when the file does not compile, for example after a careless edit. Then the compiler errors appear on the screen and the stage stops with a clear message. It does not end silently.

Run `./playbook.sh scan_sast`. Read the trace from the bottom to the top. Find three events: memory is allocated on line 8, memory is freed the first time on line 23, and memory is freed again on line 27.

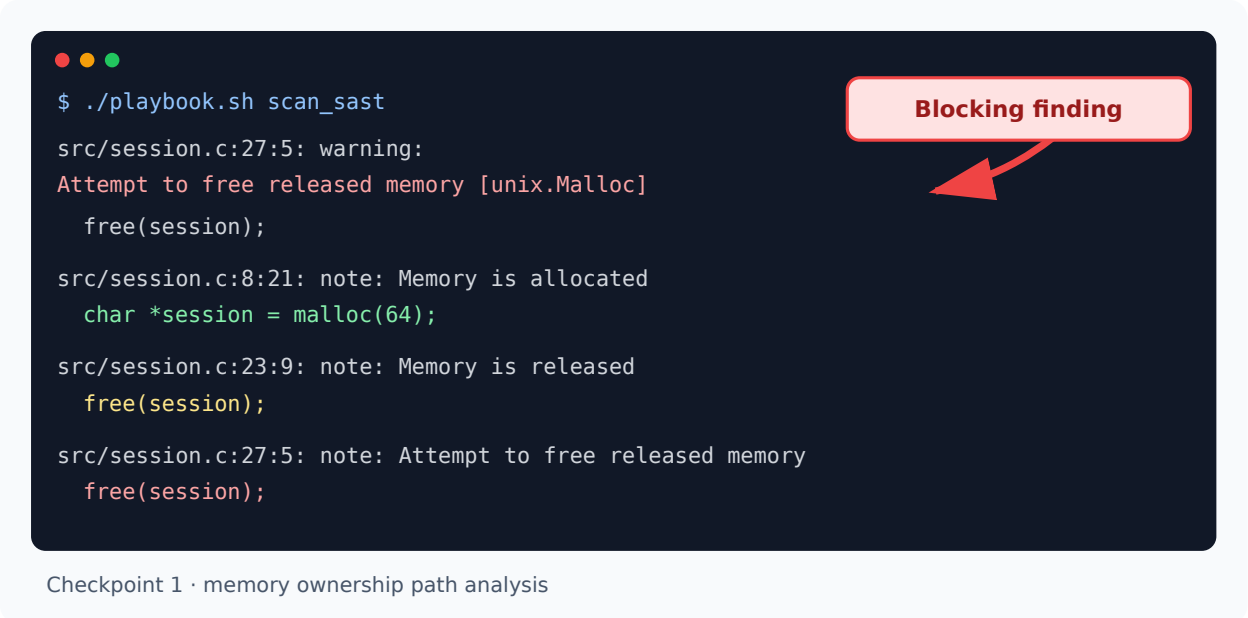


Figure 1 — Clang builds the path to the error. It does not only match text

2.1. Report in SARIF format

The text trace is easy to read, but a program cannot use it easily. So the second command in `scan_sast()` writes the same result to the file `/work/reports/sast.sarif` in the SARIF format (Static Analysis Results Interchange Format, version 2.1.0, an OASIS standard). It is JSON. GitHub Code Scanning, GitLab, Azure DevOps and other tools understand it, so you can collect reports of different analyzers in one place. `clang` creates the file with the option `-analyzer-output=sarif`, and the key `-o` sets its path. The file is not printed on the screen, so open it with `jq`:

```
jq '.version, .runs[0].tool.driver.name' /work/reports/sast.sarif
jq -r '.runs[0].results[]
  | [.ruleId, .level, .locations[0].physicalLocation.region.startLine,
     .message.text] | @tsv' /work/reports/sast.sarif
jq -r '.runs[0].results[0].codeFlows[0].threadFlows[0].locations[]
  | [.importance, .location.physicalLocation.region.startLine,
     .location.message.text] | @tsv' /work/reports/sast.sarif
```

The first command prints the format version and the name of the analyzer (`"2.1.0"` and `"clang"`). The second shows the finding in one line, with tabs between the rule, the level, the line and the message (`unix.Malloc`, `warning`, `27`, `Attempt to free released memory`). The third prints the steps of the trace with their importance and line numbers. You see the same three events: “Memory is allocated” (line 8), “Memory is released” (line 23) and “Attempt to free released memory” (line 27).

The report has nested parts: `runs` → `tool` and `results` → `codeFlows`. The table describes the main fields.

Table 2 — Main fields of the Clang SARIF report

Field	Purpose
`version`, `$schema`	Version of the format (`2.1.0`) and the address of the schema that can check the report
`runs`	List of analyzer runs. One file can join several tools. We have one run
`runs[].tool.driver`	The analyzer: `name` (`clang`), `version` (Clang version), `informationUri` (documentation)
`tool.driver.rules[]`	Description of the rules (checks). `id` and `name` are `unix.Malloc`, `fullDescription.text` says what the rule checks, `helpUri` is the help page, `defaultConfiguration.level` is the default severity
`runs[].artifacts[]`	Files that the report mentions: `location.uri` (path), `length` (size), `roles` (`resultFile` means a file with a finding)

Field	Purpose
`runs[].results[]`	List of findings. We have one: the double free
`results[].ruleId`, `ruleIndex`	Which rule fired: its ID and its number in the list `rules`
`results[].level`	Severity of the finding: `error`, `warning`, `note` or `none`. For `unix.Malloc` it is `warning`
`results[].message.text`	Message for a person: `Attempt to free released memory`
`results[].locations[]`	Place of the error. `physicalLocation.artifactLocation.uri` is the file, `region` has `startLine`, `startColumn`, `endLine`, `endColumn` (here line 27, columns 5–18)
`results[].codeFlows[]`	Path of the program to the error. `threadFlows[].locations[]` has the steps in the order of execution
`locations[].importance`	Importance of a step: `essential` is a key event, `important` is an important condition, `unimportant` is a note (for example, “Taking true branch”)
`locations[].location.message.text`	What happens in the step: memory allocation, a branch, memory release

Line and column numbers in SARIF start from one. In the lab, `uri` has the absolute path of the file inside the container. If there are no findings, the array `results` is empty, but the file is still created. It shows that the analyzer ran.

Answer before you fix the code. Why does the line `session = NULL` after the first `free()` hide the crash, but leave the ownership of the memory unclear? The right goal is to have one place where the memory is freed.

3. DAST: dynamic analysis with ASAN — 3 minutes

SAST reads the code, but it does not run it. Dynamic analysis (in SDL it is called DAST, dynamic testing) does the opposite. The program is built with extra checks and is run on real input data. Here we do not use a network scanner for web applications. We use AddressSanitizer (ASAN) from the compiler. It adds memory checks to the program and stops the program at once when it reads outside an array, uses freed memory or frees memory twice. The same tool finds leaks: when the program ends, LeakSanitizer (LSan, a part of ASAN) prints the memory that was not freed.

In `scan_dast()`, write:

```
clang -fsanitize=address -fno-omit-frame-pointer -g -Iinclude \
  src/main.c src/session.c -o /tmp/release-agent-asan
{
  echo "== run without arguments"
  /tmp/release-agent-asan || true
  echo "== run with --slot 2"
  /tmp/release-agent-asan --slot 2 || true
  echo "== run with --slot 9"
  /tmp/release-agent-asan --slot 9 || true
} 2>&1 | tee /work/reports/dast-asan.txt
```

The build uses the same `clang` as SAST. The key `-fsanitize=address` turns on ASAN. The option `-g` puts function names and line numbers in the report (`llvm-symbolizer`, which is in the image, adds the names). The program runs three times: without arguments (the normal case), with a correct quota number `--slot 2`, and with a number outside the table `--slot 9`. The part `|| true` does not let the playbook stop when ASAN ends the program with an error.

Run `./playbook.sh scan_dast`. The first two runs end without a problem (`session closed` and `quota: 30`). The third run prints the ASAN report. Find three places in it:

– the first line `ERROR: AddressSanitizer: global-buffer-overflow`, the kind of error;

– in the stack, `#0 ... in session_quota /work/repository/src/session.c:34:12` (the function, the file, the line and the column where the program reads outside the array), and `#1 ... in main`, which called this function;

– at the end, the line `SUMMARY: AddressSanitizer: global-buffer-overflow ... in session_quota`, the result for your report.



Figure 2 — The error appears only in the run with a wrong quota number

The program reads the element `quota[9]` in a table of four values. The number comes from the command line, and SAST does not know its value. This is why Clang did not find this error in section 2. The opposite is also true. The double free from section 2 did not appear here, because we did not run the program with `--reject`. Dynamic analysis sees only the paths that were executed.

Answer before you fix the code. What would ASAN show if you ran `/tmp/release-agent-asan --reject`, and why is DAST alone not enough to accept a release?

4. Search for secrets — 10 minutes

In `scan_secrets()`, write:

```
gitleaks detect --source . --config /opt/practice/gitleaks.toml \
  --log-opts=--all --report-format json \
  --report-path /work/reports/gitleaks.json --no-banner || true
```

Run:

```
./playbook.sh scan_secrets
```

Open `reports/gitleaks.json`. You should see two findings. One is the token in the current file `config/app.env`. The other is the token in the deleted file `config/legacy.env`. The option `--log-opts=--all` makes Gitleaks check the whole history. The part `|| true` keeps the training playbook running when the expected blocking result appears.



Figure 3 — Deleting a file from the working copy does not delete the secret from the commit

5. Image, SBOM and dependencies — 12 minutes

In `build_release()`, write:

```
docker build -t practice-2-release:student .
```

In `make_sbom()`:

```
syft practice-2-release:student \
  -o cyclonedx-json=/work/reports/sbom.cdx.json
```

In `scan_dependencies()`:

```
grype sbom:/work/reports/sbom.cdx.json \
  -o json --file /work/reports/grype.json || true
grype sbom:/work/reports/sbom.cdx.json --only-fixed || true
```

Run the stages one after another:

```
./playbook.sh build_release
./playbook.sh make_sbom
./playbook.sh scan_dependencies
```

The file `sbom.cdx.json` is a list of components. The file `grype.json` matches these components with a database of vulnerabilities. Count them:

```
jq '.components | length' /work/reports/sbom.cdx.json
jq '.matches | length' /work/reports/grype.json
jq -r '.matches[].vulnerability.severity' /work/reports/grype.json | sort | uniq -c
```

The second Grype command uses `--only-fixed`. It shows only the vulnerabilities that have a fixed package version in the database. For Debian 12 there may be none. Then the screen shows `No vulnerabilities found`. This does not mean that there are no vulnerabilities. The full list is in `grype.json`. The number of CVEs changes when the database is updated. Make the decision from three things: how severe the issue is, whether a fixed version exists, and whether the vulnerable component can be reached. Also notice how many build packages are in the runtime image.

6. Container — 9 minutes

In `scan_container()`, write:

```
trivy config --skip-check-update --format json \
  --output /work/reports/trivy.json . || true
trivy config --skip-check-update --severity HIGH,CRITICAL . || true
```

Run `./playbook.sh scan_container`. The original Dockerfile does not set a user without special rights. It also installs extra recommended packages. In `compose.yaml` you also see `privileged: true` and a bind mount that lets the container write to the source folder. Fix these risks even if your version of Trivy does not show them as separate findings.



Figure 4 — The SBOM answers “what do we ship”. Trivy answers “how do we run it”

Save the reports before you fix anything. The second Gate run overwrites them. You need the old numbers to compare “before” and “after”.

```
cp -r /work/reports /work/reports-before
```

7. Fixes — 6 minutes

Go to the source code: `cd /work/repository`. Use `nano`, which is installed in the image. For example, run `nano src/session.c`.

7.1. One place to free memory

In `/work/repository/src/session.c`, replace the function `handle_session`:

```
int handle_session(int reject_request) {
    int status = 0;
    char *session = allocate_session();
    if (session == NULL) {
        return 2;
    }

    if (reject_request) {
        fprintf(stderr, "request rejected: %s\n", session);
        status = 1;
        goto cleanup;
    }

    puts("session closed");

cleanup:
    free(session);
    return status;
}
```

7.2. Check of the index

At the end of `src/session.c`, add a check of the limits to the function `session_quota`:

```
int session_quota(int slot) {
    if (slot < 0 || slot >= 4) {
        return -1;
    }
    return quota[slot];
}
```

The function `main` already handles the value `-1` as `invalid slot`. Now the run with `--slot 9` prints `invalid slot`, and ASAN stays silent.

7.3. Container

Change the Dockerfile to a multi-stage build. The compiler stays in the `build` stage. Only the binary file goes to the runtime image. Install packages with `--no-install-recommends`. Run the process as a system user named `user`. Add a `HEALTHCHECK`. Replace the content of the `Dockerfile`:

```
FROM debian:12-slim AS build

RUN apt-get update \
    && apt-get install -y --no-install-recommends build-essential cmake \
    && rm -rf /var/lib/apt/lists/*
WORKDIR /src
COPY CMakeLists.txt ./
COPY include ./include
COPY src ./src
RUN cmake -S . -B build -DCMAKE_BUILD_TYPE=Release \
    && cmake --build build \
    && ctest --test-dir build

FROM debian:12-slim
RUN groupadd --system app \
    && useradd --system --gid app --create-home user
WORKDIR /app
COPY --from=build --chown=user:app /src/build/release-agent ./release-agent
USER user
HEALTHCHECK --interval=30s --timeout=3s CMD ["/app/release-agent"]
ENTRYPOINT ["/app/release-agent"]
```

In `compose.yaml`, remove `privileged` and the bind mount. Then limit the rights of the container:

```
services:
  release-agent:
    build: .
    image: practice-2-release:student
    read_only: true
    cap_drop:
      - ALL
    security_opt:
      - no-new-privileges:true
    tmpfs:
      - /tmp:size=16m,noexec,nosuid
```

7.4. Secrets and Git history

Do this step last, after you fix the code and the container. Delete `config/app.env`. Create `config/app.env.example` without the token value. Save **all** fixes in one commit. Then, after the training rotation, remove both files from the history:

```
rm config/app.env
printf 'PAYMENTS_TOKEN=\nLOG_LEVEL=info\n' > config/app.env.example
git add -A
git commit -m "fix release security findings"
FILTER_BRANCH_SQUELCH_WARNING=1 git filter-branch -f \
  --index-filter 'git rm -q --cached --ignore-unmatch config/legacy.env config/
app.env' \
  --prune-empty -- --all
rm -rf .git/refs/original
git reflog expire --expire=now --all
git gc --prune=now --quiet
```

The command `git filter-branch` works only when the working folder is clean. It may print `Cannot rewrite branches: You have unstaged changes`. This means some changes are not saved yet. Run `git add -A` and `git commit`, then repeat the cleanup block. Make sure no tokens are left in the history. The next command must print nothing:

```
git log --all -S TRAINING_TOKEN_ --oneline
```

In a real system you first revoke the real secret and replace it. Then you agree with the team how to rewrite the published history.

8. Second Security Gate run — 3 minutes

In `verify_release()`, write the calls of the functions you already filled in:

```
scan_sast
scan_dast
scan_secrets
build_release
make_sbom
scan_dependencies
scan_container
python3 /opt/practice/verify.py
```

Call the functions directly. At the start, the playbook moves to `/work/repository`. So a call to `./playbook.sh` from inside a function will not find the file. Go back to `/work` and start the check:

```
cd /work
./playbook.sh verify_release
```

The final line is:

```
[PASS] Security Gate: релиз-кандидат принят
```

The lab prints this line in Russian. It means “release candidate accepted”.

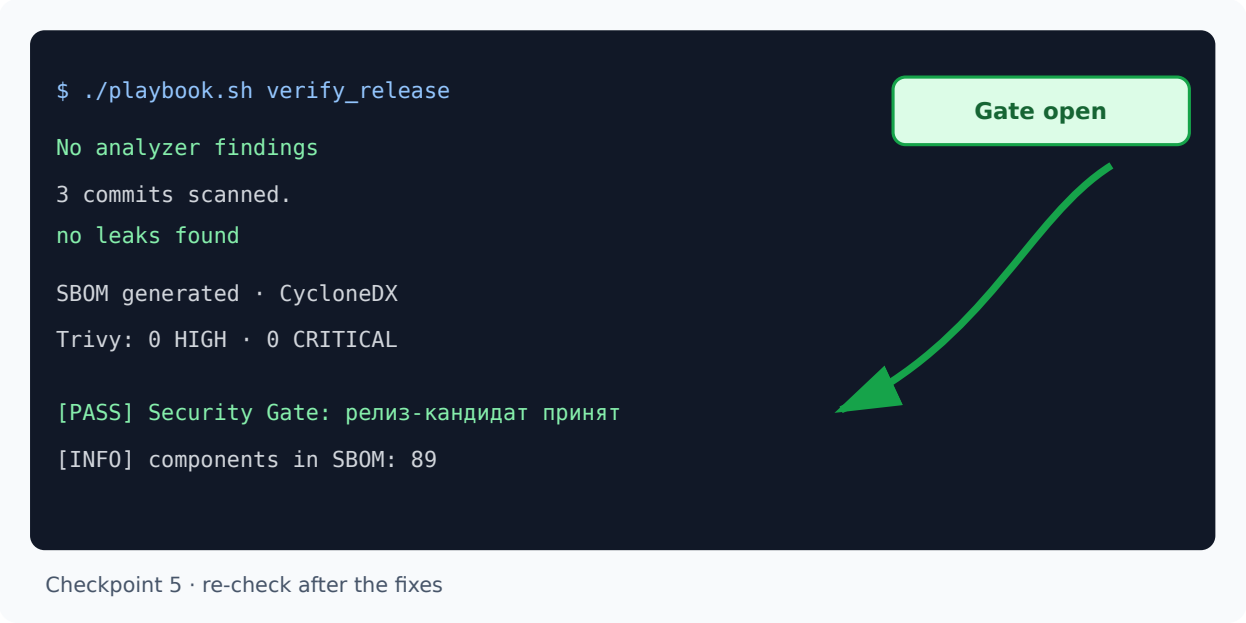


Figure 5 — The Gate opens only after the second analysis